Module 5 — Readiness Checklist

Before-class setup for the Local LLM Question Log + Bedtime Story Generator courses

Swarup Biswas (SwarupSG)

2026-07-08

Contents

Module 5 — Readiness Checklist (Before Class Setup)

Two back-to-back courses, one setup.

1. **Local LLM Question Log** (FastAPI) — everything runs **locally** on your laptop: Python, the Postgres database, and the LLM (Ollama with `llama3.2`).
2. **Bedtime Story Generator** — everything runs in the **cloud**: the Gemini API (LLM), Render (FastAPI back-end + managed Postgres), and Vercel (frontend).

This checklist is the **union of both**. Items marked **[bedtime] apply to the Bedtime course only** — do them anyway; the "why" comes later.

Windows users: you'll run the whole course inside **Ubuntu on WSL2**, *not* native Windows/PowerShell. §4 sets that up. After §4, **every command runs in your Ubuntu terminal** (the prompt ends in $).

How to use this — two parts. Part A (§1–§10) is your ticket into class: do *all* of it beforehand — accounts, the Antigravity IDE, tools, and the night-before final check. **Part B (§11) is App setup — do it only when your instructor tells you**, in class; don't clone the app code before then. Tick each box as you finish it.

> **New IDE — read this first**
>
> This course is taught in the **Google Antigravity IDE**, Google's new **agent-first (agentic) IDE** — a *separate product* from a normal code editor, with **Gemini** built into the chat panel. Google has split Antigravity into two things: the **IDE** (what you want) and a standalone **agent-manager** app (not needed here) — so grab the **IDE** specifically, from this exact page: **https://antigravity.google/product/antigravity-ide** (§5). Use only this link — don't download

from anywhere else. A regular editor (VS Code, etc.) will **not** give you the in-editor Gemini partner the course depends on.

---

1. Accounts (sign up – no installs yet)

- ☐ **GitHub** – https://github.com/signup. Pick a sensible username. **Turn on two-factor authentication** when prompted.
- ☐ **Google** – https://accounts.google.com/signup (skip if you have one). *You'll sign into Antigravity and Gemini with this account.*
- ☐ **Claude (Anthropic)** – https://claude.ai. Free tier.
- ☐ **Perplexity** – https://www.perplexity.ai. Free tier.
- ☐ **ChatGPT (OpenAI)** – https://chatgpt.com. Free tier.
- ☐ **[bedtime] Render** – https://render.com. Sign up **with your GitHub account**. *(Free Postgres on Render expires 30 days after creation – fine for the course.)*
- ☐ **[bedtime] Vercel** – https://vercel.com/signup. Sign up **with your GitHub account**. Free Hobby tier.

2. GitHub mobile app (for two-factor approval)

- ☐ Install – **iOS:** https://apps.apple.com/app/github/id1477376905 · **Android:** https://play.google.com/store/apps/det
- ☐ Sign in with your GitHub account.
- ☐ Approve **"use this device for two-factor authentication"** when the app prompts.

3. AI partner apps (install all four)

- ☐ **Claude** – https://claude.ai/download
- ☐ **ChatGPT** – https://chatgpt.com → top-right menu → Download (also App Store / Play Store)
- ☐ **Gemini** – https://gemini.google.com, sign in with Google (mobile: search "Google Gemini")
- ☐ **Perplexity** – https://www.perplexity.ai → Download

4. WSL2 + Ubuntu (Windows only)

macOS users: skip to §5.

You'll run the whole course inside **WSL2 (Ubuntu inside Windows)**. **If you took an earlier module in this programme, you already set this up – verify it and skip ahead.** Don't reinstall.

Already have WSL2/Ubuntu? Check, then skip to §5.

- ☐ Open your **Ubuntu terminal** (Start menu → **Ubuntu**; the prompt ends in `$`).
- ☐ Run `uname -a` → the output contains `Linux`.
- ☐ Run `lsb_release -d` → it shows **`Ubuntu 24.04`**.
- ☐ Both look right? **You're done with §4 – skip to §5.** *(If `lsb_release` shows a version other than 24.04 – e.g. 26.04 – message your instructor before class. The course tools target Ubuntu 24.04 / Python 3.11–3.13.)*

First time setting up WSL2? Install it now.

PowerShell is used **exactly once – right here – and never again.**

- ☐ Open **PowerShell as administrator** (search "PowerShell" → right-click → **Run as administrator**).
- ☐ Run: `wsl --install -d Ubuntu-24.04` *(The `-d Ubuntu-24.04` pins the version the course is built and tested for. Don't drop it – a newer Ubuntu ships Python 3.14, which the course tools don't support yet.)*
- ☐ **Reboot** when prompted. Ubuntu launches automatically after reboot.
- ☐ Create a **UNIX username and password** (nothing appears as you type the password – that's normal).
- ☐ Verify: `uname -a` → the output contains `Linux`.

Either way – from here on, every command runs in the Ubuntu terminal (Start menu → Ubuntu; prompt ends in `$`). If your prompt ends in `>`, that's PowerShell – switch to Ubuntu.

> Deeper WSL2 walkthrough (first launch, and the important **work in `~/code/`, not `/mnt/c/`** rule): `setup_walkthrough.md` §1.

5. Google Antigravity – the course IDE

Antigravity is Google's **agent-first IDE** with **Gemini** in the chat panel. This is where you'll write code, run commands, and get AI help all course long. Get the **IDE** (not the standalone agent-manager app).

- ☐ Get the **Antigravity IDE** from this exact page – **https://antigravity.google/product/antigravity-ide** (macOS / Linux / Windows). Use only this link; don't download Antigravity from anywhere else.
- ☐ Install (requires macOS 12+ or Windows 10+). Open it.
- ☐ **Sign in with your Google account.**
- ☐ In the right-hand chat panel, type `hello` and press Enter → confirm **Gemini replies**.
- ☐ **Windows/WSL2 only:** connect Antigravity to your Ubuntu distro so it works inside WSL2. After connecting, the bottom-left of the window shows **`WSL: Ubuntu`**. *(Steps: `setup_walkthrough.md` §7.4.)*

Optional – Zed editor with local models

*Not required, and **not a substitute for Antigravity** – the course's in-editor Gemini partner (`AGENTS.md` coaching, the "Ask Gemini" prompts, the Module 4 reveal) only works in Antigravity, so install Antigravity regardless. Zed is here purely for students who'd also like to tinker with a fast editor that can run **local models through the Ollama you install in §6.3** – nothing leaves your machine.*

- ☐ (Optional) Download **Zed** – **https://zed.dev/download** (macOS / Linux / Windows)
- ☐ (Optional) Point Zed's AI at your local **Ollama** (`llama3.2`) as the model provider – see Zed's docs: https://zed.dev/docs

6. Tools (install in your terminal)

macOS: your Terminal. Windows: your Ubuntu (WSL2) terminal from §4.

6.1 Python – 3.11, 3.12, or 3.13

*Any of **3.11 / 3.12 / 3.13** works. **Do not use 3.14** (some packages don't publish wheels for it yet, and the setup check only accepts 3.11–3.13). If you already have 3.11/3.12/3.13, **keep it – don't downgrade.***

macOS

1. `brew install python@3.12`
2. Verify: `python3 --version` → `3.11.x`, `3.12.x`, or `3.13.x`

Ubuntu / WSL2

1. `sudo apt update && sudo apt install -y python3 python3-venv python3-pip`
2. Verify: `python3 --version` → `3.11.x`, `3.12.x`, or `3.13.x` *(Ubuntu 24.04 ships 3.12 by default)*

PEP 668: Ubuntu blocks `pip install` outside a virtual environment by design – you'll always work inside a venv, exactly what the course teaches.

6.2 Postgres 17

Postgres 16 or 18 also work; 17 is what the course assumes.

macOS

1. `brew install postgresql@17`
2. `brew services start postgresql@17`
3. `psql -d postgres -c "CREATE USER postgres WITH PASSWORD 'postgres' SUPERUSER;"`

4. Verify: `pg_isready -h localhost` → `accepting connections`

Ubuntu / WSL2 – run one step at a time:

1. `sudo apt install -y postgresql-common`
2. `sudo /usr/share/postgresql-common/pgdg/apt.postgresql.org.sh` *(interactive – press **Enter** when prompted)*
3. `sudo apt install -y postgresql-17`
4. `sudo systemctl start postgresql && sudo systemctl enable postgresql`
5. `sudo -u postgres psql -c "ALTER USER postgres WITH PASSWORD 'postgres';"`
6. Verify: `pg_isready -h localhost` → `accepting connections`

6.3 Ollama + the `llama3.2` model

macOS

1. Download the `.dmg` from https://ollama.com/download (needs macOS 14 Sonoma+), drag to Applications, open it.
2. `ollama pull llama3.2` (~2 GB download)

Ubuntu / WSL2

1. Install: `curl -fsSL https://ollama.com/install.sh | sh`
2. Start: `sudo systemctl start ollama && sudo systemctl enable ollama`
3. `ollama pull llama3.2` (~2 GB download)

Both – verify:

```
curl -s -X POST http://localhost:11434/api/chat \
  -H "Content-Type: application/json" \
  -d '{"model":"llama3.2","messages":[{"role":"user","content":"hi"}],"stream":false}' | head
   ↪  -3
```

→ a JSON response with a `message` field.

GPU note (WSL2): the default install is CPU-only, which is fine for the course. Optional Windows-host GPU setup is in `setup_walkthrough.md` (Appendix).

6.4 Git

☐ **macOS & Ubuntu:** usually pre-installed. Verify: `git --version`. *(If missing on Ubuntu: `sudo apt install -y git`.)*

6.5 GitHub CLI (`gh`) – Antigravity uses it to push and pull

macOS

1. `brew install gh`
2. Verify: `gh --version`

Ubuntu / WSL2 – install from the official GitHub CLI repository (copy the whole block):

```
sudo mkdir -p -m 755 /etc/apt/keyrings \
  && wget -qO- https://cli.github.com/packages/githubcli-archive-keyring.gpg \
     | sudo tee /etc/apt/keyrings/githubcli-archive-keyring.gpg > /dev/null \
  && sudo chmod go+r /etc/apt/keyrings/githubcli-archive-keyring.gpg \
  && echo "deb [arch=$(dpkg --print-architecture)
   ↪  signed-by=/etc/apt/keyrings/githubcli-archive-keyring.gpg]
   ↪  https://cli.github.com/packages stable main" \
     | sudo tee /etc/apt/sources.list.d/github-cli.list > /dev/null \
  && sudo apt update && sudo apt install -y gh
```

Then verify: `gh --version`. *(If this block errors, see `setup_walkthrough.md` – the gh keyring path is a known 2026 gotcha.)*

7. Connect Antigravity to GitHub (so Gemini can push/pull from chat)

- ☐ Open Antigravity's integrated terminal – **View** → **Terminal**, or `Ctrl+`` (backtick). *Windows/WSL2: confirm it's your Ubuntu terminal (prompt ends in $').*
- ☐ Run: `gh auth login`
- ☐ Answer in **this exact order:** GitHub.com → **HTTPS** → **Yes** (authenticate Git) → **Login with a web browser**
- ☐ Copy the **8-character one-time code** it prints (e.g. XXXX-XXXX).
- ☐ In the browser tab that opens (https://github.com/login/device – open it manually if it doesn't), paste the code → **Continue** → approve the permissions.
- ☐ Your phone buzzes → open the **GitHub mobile app** → tap **Approve**.
- ☐ Back in the terminal you should see ✓ `Authentication complete` and ✓ `Configured git protocol`.

Confirm Gemini can push code from chat

- ☐ Create a sandbox:

```
mkdir ~/antigravity-test && cd ~/antigravity-test
echo "hello" > note.txt
git init && git add . && git commit -m "first"
```

- ☐ Open the `antigravity-test` folder in Antigravity (**File** → **Open Folder**).
- ☐ In the Gemini chat panel, paste exactly: > Add a line 'world' to note.txt, commit the change, create a new public GitHub repo for this folder under my account, and push.
- ☐ Gemini should **run** `gh repo create`, `git remote add`, and `git push -u origin main` **in the terminal** – not just print them as text.
- ☐ Open your GitHub profile in a browser → confirm the `antigravity-test` repo exists with **two commits**.
- ☐ **If Gemini only prints the commands instead of running them, message the instructor before class.**
- ☐ (Optional cleanup) `gh repo delete <yourusername>/antigravity-test --yes`

Keep going with §8–§9 below (the [bedtime] bedtime cloud accounts). **Don't clone the app repos yet** – that's **Part B (§11)**, which you'll do *when your instructor tells you*, in class.

8. [bedtime] Gemini API key (created together in class – reference only)

Bedtime course only. Done live in the first bedtime session – here for reference.

1. Open https://aistudio.google.com/apikey
2. Sign in with the **same Google account** you used for Antigravity.
3. Accept the Terms of Service. AI Studio creates a default Google Cloud project and an API key in one step – no billing setup needed for free-tier use.
4. Copy the key (it starts with `AIza...`).
5. Save it in a **password manager**. **Do not** paste it into chats, screenshot it, or commit it to a public repo.
6. Verify it works (substitute `<YOUR_KEY>`):

```
curl -s "https://generativelanguage.googleapis.com/v1beta/models?key=<YOUR_KEY>" | head
 ↪  -5
```

→ a JSON list of models.

7. Free-tier rate limits for your account: https://aistudio.google.com/rate-limit

9. [bedtime] Connect Render and Vercel to GitHub

Bedtime course only.

Render ↔ GitHub

1. Sign in – https://dashboard.render.com
2. Top right: **+ New** → **Web Service**
3. Click the **GitHub** card → approve the OAuth permissions
4. Back out (no deploy yet)

Vercel ↔ GitHub

1. Sign in – https://vercel.com/dashboard
2. Top right: **Add New** → **Project**
3. Click **Install** under the GitHub heading → approve the OAuth permissions
4. Back out (no project yet)

10. Final check (the night before the first session)

Run all four – this confirms your **environment** is ready. **All must pass.** *(Windows: run these in your Ubuntu terminal.)*

- ☐ `python3 --version` → `3.11.x`, `3.12.x`, or `3.13.x`
- ☐ `pg_isready -h localhost` → `accepting connections`
- ☐ Ollama smoke curl (from §6.3) → JSON response with a `message` field
- ☐ Open **Antigravity**, type `hello` in the Gemini chat panel → Gemini replies

*(The full 8-check `verify_setup.sh` gate runs against the app repo in **Part B (§11)** – you'll do that once your instructor directs you to clone.)*

Part B – App setup (do this ONLY when your instructor tells you)

Everything above (§1–§10) is your before-class checklist. **Don't start the steps below until directed** – you'll clone and set up the app code together, in class.

Both courses build a **single-turn LLM app** (one question in, one answer out): the **Local LLM Question Log** runs locally on your laptop; the **[bedtime] Bedtime Story Generator** runs in the cloud. Same shape, two deployment stories.

11. App setup

First, pick a working folder. *(Windows/WSL2: inside Ubuntu, use `~/code/` – **NOT** `/mnt/c/...`; Python there is 10–50× slower.)*

```
mkdir -p ~/code && cd ~/code
```

A. Local LLM Question Log (FastAPI) – clone, database, verify

- ☐ Clone it (your phone buzzes – approve in the GitHub mobile app):

```
git clone https://github.com/SwarupSG/fastapi-ollama-postgres-cohort.git
cd fastapi-ollama-postgres-cohort
```

- ☐ Create the database: `createdb llm_question_log`
- ☐ Apply the schema: `psql -d llm_question_log -f sql/001_create_interactions.sql`

- ☐ Run the gate: `./scripts/verify_setup.sh` → **8 green checks**, ending with *"All checks passed. You're ready for Module 1." (Everyone runs the `.sh` – Windows students run it inside Ubuntu. There is no `.ps1`.)*
- ☐ Open the folder in Antigravity → type `what's in this folder?` in Gemini → it replies with a directory listing.

B. [bedtime] Bedtime Story Generator – clone

- ☐ From the same parent folder:

```
cd ~/code
git clone https://github.com/SwarupSG/bedtime-story-generator-cohort.git
cd bedtime-story-generator-cohort
ls
```

- ☐ Expected: `app/`, `dist/`, `frontend/`, `scripts/`, `README.md`, `AGENTS.md`, etc.
- ☐ **No further pre-class action** – the bedtime app's database, `.env`, Gemini key, and Render/Vercel deploy are all done live in the bedtime sessions.

---

12. If anything fails

1. **Paste the failing command and its full error into an AI app** – use whichever you're comfortable with (Claude, ChatGPT, Gemini, or Perplexity – you installed all four in §3). Most setup errors (Homebrew, apt, WSL2, Postgres) are common, and an AI will walk you through the fix. *Once Antigravity is set up, prefer its built-in Gemini for anything repo-specific – it loads `AGENTS.md` at the cohort root, so it's **course-aware** and knows this course's exact friction points. But don't wait on that if you haven't used Antigravity yet – any AI app is fine for setup errors.*
2. If the AI's fix doesn't work, post a **screenshot** of the failing command and error in the cohort **async help channel**.
3. The live session is the **last resort** – arrive ready.